DSA Master Revision Sheet (V2)

ALL APPROACHES INCLUDED • DAYS 01 - 03

Bhai, tere bolne par maine is sheet ko full-detailed bana diya hai. Ab har ek question ke saare possible approaches (Brute Force, Better, aur Optimal) proper codes aur time/space complexity ke saath listed hain taaki interview mein jab interviewer "Can you optimize it further?" bole, toh tere paas saare jawab ready hon!

1. Two Sum (LeetCode 1)

Problem: Array mein do elements dhoondne hain jinka sum `target` ke equal ho, aur unke indices return karne hain.

Approach 1: Brute Force BRUTE FORCE

Two nested loops ka use karke har ek pair ka sum check karo.

```
def twoSumBrute(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

> **Time Complexity:** O(N²) — Do nested loops poore array ko scan karte hain.
> **Space Complexity:** O(1) — Koi extra space use nahi hoti.

Approach 2: Optimal (Hash Map) OPTIMAL

Single pass linear scan ke saath complement (`target - current`) track karte chalna.

```
def twoSumOptimal(nums, target):
    num_map = {} # value -> index
    for i, num in enumerate(nums):
        remaining = target - num
        if remaining in num_map:
            return [num_map[remaining], i]
        num_map[num] = i
    return []
```

> **Time Complexity:** O(N) — Array ka single traversal aur Hash Map lookup O(1) hota hai.
> **Space Complexity:** O(N) — Map mein elements store karne ke liye space chahiye.

2. Sort Colors (LeetCode 75)

Problem: 0s, 1s, aur 2s ke mix array ko in-place bina built-in function ke sort karna hai.

Approach 1: Brute Force (Sorting) BRUTE FORCE

Kisi bhi standard sorting function (jaise Merge Sort ya Quick Sort) ka use karke array sort kar do.

```
def sortColorsBrute(nums):
    nums.sort() # Internal standard sorting algorithms
```

> **Time Complexity:** O(N log N) — Standard sorting complexity.
> **Space Complexity:** O(1) ya O(N) — Depends on sorting algorithm implementation.

Approach 2: Better (Counting Sort) BETTER

Array ko ek baar read karke 0s, 1s, aur 2s ka individual count nikaal lo, fir doosri baar mein array par overwrite kar do.

```
def sortColorsBetter(nums):
    count0, count1, count2 = 0, 0, 0
    for num in nums:
        if num == 0: count0 += 1
        elif num == 1: count1 += 1
        else: count2 += 1

    for i in range(len(nums)):
        if i < count0: nums[i] = 0
        elif i < count0 + count1: nums[i] = 1
        else: nums[i] = 2
```

> **Time Complexity:** O(N) + O(N) = O(2N) → O(N) — Do baar linear passes lagte hain.
> **Space Complexity:** O(1) — Sirf 3 variables frequency counter ke liye lagte hain.

Approach 3: Optimal (Dutch National Flag Algorithm) OPTIMAL

Three pointers (low, mid, high) ka use karke pure array ko single pass mein in-place sort karna.

```
def sortColorsOptimal(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
```

> **Time Complexity:** O(N) — Single pass execution.
> **Space Complexity:** O(1) — Strictly constant space sorting.

3. Majority Element (LeetCode 169)

Problem: Array ka woh element find karo jo > N/2 baar aaya hai.

Approach 1: Brute Force BRUTE FORCE

Nested loops se har element ke liye array mein uski total frequency check karna.

```
def majorityElementBrute(nums):
    n = len(nums)
    for i in range(n):
        count = 0
        for j in range(n):
            if nums[j] == nums[i]: count += 1
        if count > n // 2: return nums[i]
    return -1
```

> **Time Complexity:** O(N²) — Nested linear scanning.
> **Space Complexity:** O(1) — Koi auxiliary database coding nahi.

Approach 2: Better (Hash Map / Frequency Counter) BETTER

Dictionary ka use karke har element ka occurrence count maintain karna (Garvit's active approach).

```
def majorityElementBetter(nums):
    counts = {}
    n = len(nums)
    for num in nums:
        counts[num] = counts.get(num, 0) + 1
        if counts[num] > n // 2:
            return num
```

> **Time Complexity:** O(N) — Single loop verification.
> **Space Complexity:** O(N) — Unique elements tracking frequency buffer storage.

Approach 3: Optimal (Boyer-Moore Voting Algorithm) OPTIMAL

Is algorithm mein hum ek potential element aur uske vote count balance ko handle karte hain. Jab same element mile count badhao, alg mile toh ghatao. Kyunki majority element strictly > N/2 hai, aakhri mein wahi bachega.

```
def majorityElementOptimal(nums):
    candidate = None
    count = 0
    for num in nums:
        if count == 0:
            candidate = num
        count += (1 if num == candidate else -1)
    return candidate
```

> **Time Complexity:** O(N) — Single linear loop pass.
> **Space Complexity:** O(1) — Zero mapping space consumption. Strictly optimal.

4. Best Time to Buy and Sell Stock (LeetCode 121)

Problem: Kisi single day par buy karke upcoming best future day par sell karke max profit nikaalna.

Approach 1: Brute Force BRUTE FORCE

Nested loop se saare standard valid buy-sell transactions calculate karke unka maximum save karna.

```
def maxProfitBrute(prices):
    max_prof = 0
    n = len(prices)
    for i in range(n):
        for j in range(i + 1, n):
            profit = prices[j] - prices[i]
            if profit > max_prof: max_prof = profit
    return max_prof
```

> **Time Complexity:** O(N^2) — Double loops complexity.
> **Space Complexity:** O(1) — No extra data allocations.

Approach 2: Optimal (One-pass Greedy Approach) OPTIMAL

Array ghumte waqt lowest purchase valuation (min_price) maintain karo, aur uske linear index comparison se instantaneous standard max profit update karo.

```
def maxProfitOptimal(prices):
    min_price = float('inf')
    max_profit = 0
    for price in prices:
        if price < min_price:
            min_price = price
        elif price - min_price > max_profit:
            max_profit = price - min_price
    return max_profit
```

> **Time Complexity:** O(N) — Strictly linear traversal scan.
> **Space Complexity:** O(1) — Zero memory usage scales.